Crop Disease Detector

Project Proposal – Track A: Application Development

Member 1 (ID) Member 2 (ID) Member 3 (ID)

GitHub: https://github.com/your-username/crop-disease-detector

October 10, 2026

1 Problem Statement

Farmers, especially in developing regions, often lack access to agricultural experts who can quickly and accurately diagnose plant diseases. By the time a disease is identified through traditional means, it has frequently spread beyond the point of effective treatment, leading to significant crop loss and economic hardship. There is a clear need for an accessible, instant, and trustworthy diagnostic tool that not only identifies the disease but also tells the farmer what to do about it.

This project builds a Crop Disease Detector, an AI-powered web application that lets a farmer upload a photo of a diseased leaf and instantly receive:

1. The predicted disease (with confidence score).
2. A visual explanation (Grad-CAM heatmap) showing which parts of the leaf the model focused on.
3. Actionable treatment recommendations: specific fungicides, dosage, organic alternatives, and preventive measures tailored to the diagnosed disease.

2 Dataset

PlantVillage Dataset (Kaggle)

- ∼54,000 labeled leaf images.
- 38 disease classes across multiple crop species (tomato, potato, pepper, apple, grape, corn, etc.).
- Clean, well-documented, and widely used as a benchmark.

Known limitation (which we address): the dataset contains mostly lab-condition images with uniform backgrounds. Models trained on it can appear highly accurate while failing on real-world field photos. Our robustness testing (Section 5) explicitly investigates this gap.

3 Model and Architecture

Base model: fine-tuned ResNet50 (or EfficientNet-B0) via transfer learning.

Why transfer learning:

- Pretrained ImageNet weights provide strong low-level feature extractors (edges, textures, color gradients).

- Fine-tuning only the later layers substantially reduces training time and data requirements.
- Reported in the literature to reach high accuracy on PlantVillage with modest compute.

Training pipeline:

- Input: 224×224 RGB images.
- Augmentation: random flips, rotations, color jitter, RandAugment.
- Loss: cross-entropy with class weights (to handle class imbalance).
- Optimizer: AdamW with cosine annealing.
- Early stopping on validation F1.

4 Core Differentiator: Explainability and Treatment Engine

Most student projects stop at "predicted class + confidence." Ours goes two steps further.

4.1 Grad-CAM Visual Explanations

Grad-CAM generates a heatmap overlaid on the input image, highlighting the regions the model considered most important for its prediction.

- **Builds trust:** the user can see whether the model is looking at the diseased spot rather than the background.
- **Supports bias analysis:** reveals whether the model learns dataset artifacts (e.g., uniform backgrounds) instead of disease patterns.
- **Debugging tool:** irrelevant highlighted regions signal the need for retraining or better augmentation.

Implementation: `pytorch-grad-cam` or `captum`, integrated into the Streamlit UI.

4.2 Treatment Recommendation Engine

After diagnosis, the app displays a structured treatment plan for the detected disease, covering all 38 classes.

Field	Example (Early Blight – Tomato)
Cause	Fungus *Alternaria solani*
Chemical treatment	Chlorothalonil or Mancozeb spray, every 7–10 days
Organic alternative	Neem oil spray, copper-based fungicide
Immediate action	Remove and destroy infected leaves; avoid overhead watering
Prevention	Crop rotation, resistant varieties, proper spacing for airflow
Severity guidance	If $> 30\%$ of leaf area is affected, consider removing the plant

Table 1: Example treatment record. Final content will be verified against cited sources.

Implementation approach:

- **MVP:** curated JSON mapping each of the 38 classes to treatment info, sourced from agricultural extension documents (FAO, university guides).
- **Stretch:** RAG pipeline over agricultural PDFs using LangChain and a vector database for more context-aware advice.

5 Real-World Robustness Testing

The problem: PlantVillage's clean lab images do not represent real field conditions: soil, shadows, multiple leaves, blur, and varying lighting.

What we will do:

1. Collect ∼100 real-world field images (PlantDoc dataset, iNaturalist, or similar).
2. Run the model on them and record the accuracy drop (hypothesis: a substantial drop relative to lab-condition accuracy).
3. Apply mitigations: test-time augmentation, background removal (`rembg`), or fine-tuning on a small field-image set.
4. Report before/after results.

6 User Interface (Streamlit)

Flow:

1. User opens the hosted web app.
2. User uploads a leaf photo (drag-and-drop or camera capture on mobile).
3. The app displays: predicted disease and confidence; Grad-CAM heatmap beside the original image; a structured treatment plan (chemical, organic, immediate, preventive); and a warning if confidence is below 70%.
4. Optional: download the diagnosis as a PDF report.

Hosting: Streamlit Community Cloud (frontend) with the model hosted on Hugging Face Hub or GitHub Releases. No localhost-only evaluation.

7 Evaluation and Bias Check

Metrics: overall accuracy, macro F1, per-class precision/recall, 38×38 confusion matrix, top-3 accuracy.

Bias check:

- Per-class performance breakdown to identify weak diseases.
- Class imbalance analysis: does the model favor majority classes?
- Grad-CAM analysis: does the model focus on disease regions or background artifacts?
- Robustness gap: lab vs. field image performance.

8 Tech Stack

9 Deliverables

1. GitHub repository: clean code, commit history showing individual contributions, README with setup instructions.

Layer	Tool
Model training	PyTorch, torchvision
Explainability	pytorch-grad-cam / captum
UI	Streamlit
Treatment data	Curated JSON (MVP) / LangChain + ChromaDB (stretch)
Deployment	Streamlit Community Cloud + Hugging Face
Version control	GitHub (datasets excluded via `.gitignore`)
Report	LaTeX

2. Hosted web application with a live URL.
3. LaTeX technical report: methodology, architecture diagrams, Grad-CAM analysis, robustness findings, challenges faced.
4. Treatment knowledge base (JSON or RAG pipeline), documented.

10 Team Roles

Member	Primary responsibility
Member 1	Data pipeline, preprocessing, model training and evaluation
Member 2	Grad-CAM, treatment knowledge base, robustness testing
Member 3	Streamlit UI, deployment, report assembly

Table 2: Initial split; all members review and understand the full codebase.

11 Timeline

Step	Task	Days
1	Setup, dataset download, EDA	1–2
2	Preprocessing, augmentation pipeline	1–2
3	Baseline ResNet50 training	2–3
4	Fine-tuning and hyperparameter search	2–4
5	Grad-CAM integration	1–2
6	Treatment engine (JSON + UI cards)	1–2
7	Robustness testing on field images	2–3
8	Streamlit UI and deployment	1–2
9	Evaluation, bias check, LaTeX report	2–3
–	Buffer for debugging	2–3

Table 3: Tentative schedule; adjusted to the instructor's deliverable dates.

12 Risks and Mitigations

Risk	Mitigation
Model overfits to clean backgrounds	Aggressive augmentation and robustness testing
Class imbalance skews results	Weighted loss and per-class metrics
Deployment issues	Streamlit Cloud (low-config) with model on Hugging Face
Treatment information accuracy	Source from FAO / university extension documents; cite sources
Time overrun	MVP first (classifier + Grad-CAM + JSON treatments), then stretch goals